

18-Marks

UNIT IV

CHAPTER 4

Database Transactions Management

Syllabus

Basic concepts of a Transaction, Transaction Management, Properties of Transactions, Concept of Schedule, Serial Schedule, Serializability: Conflict and View, Cascaded Aborts, Recoverable and Non-recoverable Schedules, Concurrency Control: Need, Locking Methods, Deadlock handling and Time-stamp based Protocols.

4.1	Basic Concepts of Transaction	4-3
4.1.1	Process of Transaction	4-3
GQ.	Explain Read and Write operations of Transaction (5 Marks).....	4-3
4.2	Transaction States	4-3
GQ.	Explain States of Transaction. (5 Marks)	4-3
4.3	ACID Properties	4-4
GQ.	What are ACID properties of a transaction ?	4-4
4.4	Implementation of Atomicity, Durability and Isolation	4-5
GQ.	Write an example of implement Atomicity, Consistency, isolation and durability.	4-5
4.5	Transaction Management	4-5
GQ.	Explain Commit and Rollback operations of Transaction. (5 Marks)	4-5
4.6	Concept of Schedule, serial Schedule	4-6
4.6.1	Schedule	4-6
GQ.	Write note on Schedule. (5 Marks).....	4-6
4.6.2	Types of Schedule	4-6
4.6.2(A)	Serial Executions (Schedule).....	4-6
GQ.	Explain Serial Execution. (5 Marks)	4-6
4.6.2(B)	Concurrent Executions (Schedule)	4-7
GQ.	Explain concurrent Execution. (5 Marks)	4-7
GQ.	Explain advantages of concurrent execution of transactions. (5 Marks).....	4-7
GQ.	Write note on problems occurred in concurrent Execution of Transaction. (10 Marks)	4-8
4.7	Serializability	4-9
GQ.	Explain the term : Serializability. (10 Marks)	4-9
4.8	Types of Serializability	4-11
GQ.	Explain types of Serializability. (10 Marks).....	4-11
4.8.1	Conflict Serializability	4-11
4.8.2	View Serializability	4-13
4.9	Recoverability : Recoverable and Non-recoverable Schedules, Cascaded Aborts	4-13
4.9.1	Recoverable Schedule.....	4-13

GQ. Explain Recoverability. (10 Marks)	4-13
4.9.2 Non Recoverable Schedule	4-14
4.9.3 Cascading Schedule	4-14
4.9.4 Cascaded Aborts	4-14
4.9.5 Cascadeless Schedule	4-15
4.9.6 Strict Schedule	4-15
4.10 Concurrency Control : Need	4-15
GQ. Explain concurrency control in distributed database.	4-15
4.10.1 Need of Concurrency Control	4-15
4.11 Locking Methods	4-16
GQ. Explain different types of Locking Methods. (10 Marks)	4-16
4.12 Different Concurrency Control Protocols	4-17
4.12.1 Lock Based Concurrency Control	4-17
GQ. Explain different two phase locking protocols in transaction management. (5 Marks)	4-17
4.12.2 Time-Stamp Based Concurrency Control	4-18
GQ. Explain the Time Stamp based concurrency control. (10 Marks)	4-18
4.12.3 Optimistic Concurrency Control (OCC)	4-19
GQ. Explain the Optimistic Concurrency Control. (10 Marks)	4-19
4.13 Deadlock	4-19
GQ. When deadlock occurs? How to prevent them? How to recover if dead lock takes place? (10 Marks)	4-19
4.13.1 Deadlock Prevention	4-19
4.13.2 Deadlock Detection	4-20
4.13.3 Deadlock Recovery	4-21
4.14 Recovery System	4-21
4.14.1 Failure Classification	4-21
GQ. Write note on Failure Classification. (5 Marks)	4-21
4.14.2 Storage Structure	4-21
GQ. Explain Storage structure in recovery system. (5 Marks)	4-21
4.14.3 Recovery and Atomicity	4-22
GQ. Write note on Recovery and Atomicity. (5 Marks)	4-22
4.15 Log based Recovery	4-22
GQ. Explain log based recovery scheme. (8 Marks)	4-22
4.16 Checkpoints	4-23
GQ. Write note on checkpoints. (5 Marks)	4-23
4.17 Shadow Paging	4-23
GQ. Write note on Shadow Paging. (5 Marks)	4-23
4.18 ARIES Algorithm	4-24
GQ. Describe the three phases of the ARIES recovery method.	4-24
GQ. What are the roles of the analysis, Redo and Undo phases in ARIES ? Explain with suitable example.	4-24
GQ. Describe the ARIES recovery algorithm with example.	4-24
❖ Chapter Ends	4-27

4.1 BASIC CONCEPTS OF TRANSACTION

- A **transaction** is a series of operations performed as a single logical unit of work on the Database Management System. Transaction leads to modification in the database contents.
- A transaction is initiated by a user program written in the high level data manipulation languages like SQL or programming language like java. The transaction consists of all the operations executed between begin transaction and end transaction.
- Transactions in a database environment have two main purposes :

- (1) To provide reliable units of work that allow correct recovery from failures and keep a database consistent even in cases of system failure, when execution stops (completely or partially) and many operations upon a database remain uncompleted, with unclear status.

For example, in a banking System, transferring money from one account to another leads to change in balance values of both accounts. To keep the database in consistent state both changes must succeed or fail together.

- (2) To provide isolation between programs accessing a database concurrently. If this isolation is not provided, the programs' outcomes are possibly incorrect.

4.1.1 Process of Transaction

Q. Explain Read and Write operations of Transaction. (5 Marks)

- The execution of transaction leads to perform series of read and write operations on database objects, which are explained below :

(A) Read Operation

To perform transaction on a data which is present in main memory Read () operation is used.

Read (X);

This statement states that value of X is read from main memory (i.e. from local buffer).

If it is not present in the main memory then :

- (1) It is first brought into main memory from disk, using Input () operation.

Like Input (X);

- (2) Then its value is copied into a program variable as shown in Fig. 4.1.1.

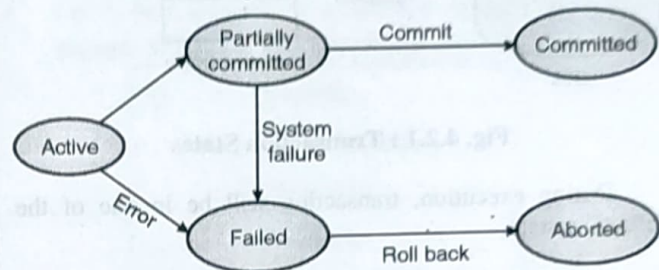


Fig. 4.1.1 : Read Operation

(B) Write Operation

To write a database object, an in-memory copy of the object is first modified and then written to disk.

Write (X);

This statement states that value of X is written to local buffer which is in main memory. And then this updated value is copied into appropriate disk block, so that the changes are done permanently as shown in Fig. 4.1.2.

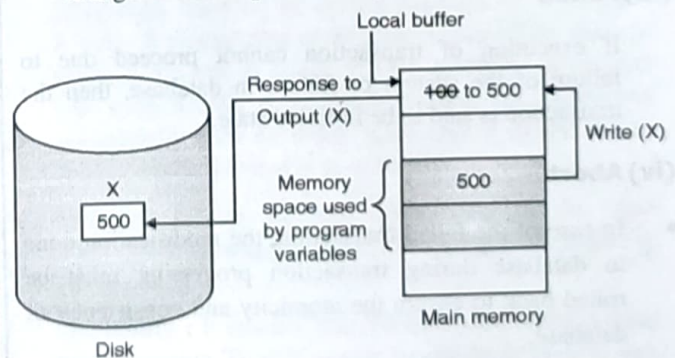


Fig. 4.1.2 : Write Operation

4.2 TRANSACTION STATES

Q. Explain States of Transaction. (5 Marks)

There are five states of transaction

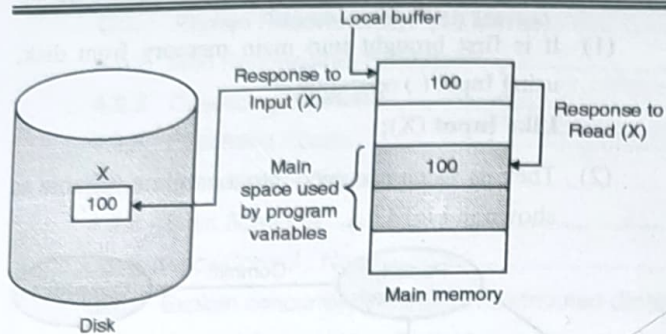


Fig. 4.2.1 : Transaction States

During execution, transaction will be in one of the following state.

(i) Active

- This is the first state of transaction. In this state the transaction is being executed.
- This state is entry point to every transaction. Transactions which are in active state indicate that their execution has been started.

(ii) Partially Committed

When a transaction completes all operations, it will enter into partially committed state. Even the last operation has been performed; data is still not saved to the database.

(iii) Failed

If execution of transaction cannot proceed due to failure of the system or failure in database, then the transaction is said to be in failed state.

(iv) Abort

- In case of the failed transaction, the modification done in database during transaction processing must be rolled back to ensure the atomicity and consistency of database.
- The transaction enters into 'Abort' this state after roll back operation is performed. It is the end of transaction when any fault occurs.

(v) Committed

- If without any error transaction completed successfully it will come into committed state which will allow to made changes permanent into database. This is the last step of a transaction, if it executes without fail.

- Every transaction goes through at least three states among of five. Either transaction may goes through active - partially committed - committed or through active - Failed - aborted, or through active - partially committed - failed - abort.

4.3 ACID PROPERTIES

Q. What are ACID properties of a transaction ?

- In computer science, **ACID** (Atomicity, Consistency, Isolation, Durability) is a set of properties of database transactions. In the context of databases, a sequence of database operations that satisfies the ACID properties and thus, can be perceived as single logical operation on the data is called a **transaction**.

- **ACID properties** are explained in detail as follows :

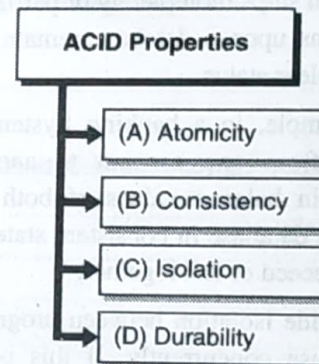


Fig. 4.3.1 : ACID Properties

(A) Atomicity

- Atomicity is based on the concept that each transaction be "all or nothing" : if any one part of the transaction in a sequence fails, then the entire transaction fails, and there will be no change in database state.
- An atomic system must guarantee atomicity in every situation, including power failures, errors and crashes. For the outside world, a committed transaction appears completely by its effects on the database. That means it should be indivisible ("atomic"), and an aborted transaction does not happen.
- This property states that, either all operations contained by a transaction are done successfully or none of them complete at all. To maintain the consistency of data this property is very useful.

✓ { Read(X) - database to local
write(X) - local buffer to
database.

► (B) Consistency

- The consistency property ensures that the transaction executed on the database system will bring the database from one valid state to another.
- The data which is written by the transaction must be valid according to the all standard rules and regulations regarding constraints, cascades, triggers etc. Consistency does not guarantee accuracy of the transaction as per the expectations of programmer.

► (C) Isolation

- When transactions are performed in a sequence, the state of a system is always valid without any problem. But sometimes we may have to perform multiple transactions concurrently.
- In case of concurrent transactions, the isolation property ensures that the system state should be same that would be obtained if transactions were executed sequentially, i.e., one after the other. The effect of any incomplete transaction should be invisible to other transaction by means of isolation.

► (D) Durability

- The durability property assures that after transaction committed successfully the updates made should remain permanent in the database even in the event of power loss, crashes or errors.
- For example in a database management system, when a series of transaction is executed, the modification done should be stored permanently even if the database crashes just after the transaction. To avoid problem because of power loss, the states of database after every transaction must be recorded in a non-volatile memory.

4.4 IMPLEMENTATION OF ATOMICITY, DURABILITY AND ISOLATION

GQ. Write an example of implement Atomicity, Consistency, isolation and durability.

A complete example of transaction

Consider a banking system having number of accounts stored in the database.

To access the database following operations are used in transaction.

- Read(X), which transfer data X from database to local buffer & Write(X), which transfer the data X from local buffer to database.
- Suppose, before the transaction M has \$500 and N has \$1000 balance in their accounts.
- Let T_i be a transaction which will transfer \$150 from account M to N, this will be written as :

```
Ti :  Read(M);
      M := M - 150;
      Write(M);
      Read(N);
      N := N + 150;
      Write(N);
```

- **Atomicity** : If any failure occurs in the system during transaction proceeded up to Write (M); and further operations does not executed due to failure occurrence, it will leads the banking database system in inconsistent state if atomicity property is not provided. But if the atomicity property is provided then if all operations are not executed, then the operations which are executed before failure get cancelled.
- **Consistency** : If consistency is provided, then in above example the sum of balance in account M and N will be same before transaction T_i perform and after T_i completes. That is, in initial situation $M + N$ gives \$1500. If transaction committed successfully then M holds \$350 and N holds \$1150. Addition of amount will give \$1500. If transaction does not committed successfully then M holds \$500 and N holds \$1000. Addition of amount will give \$1500.
- **Isolation** : This property takes care of no other transactions should interfere with transaction T_i during its execution.
- **Durability** : It ensures that the data modification done by transaction T_i will persist in database even if any failure occurs during system after committing the transaction.

4.5 TRANSACTION MANAGEMENT

GQ. Explain Commit and Rollback operations of Transaction. (5 Marks)

To ensure database integrity it is important to control transactions. Transactions can be controlled using TCL commands.

Unit
IV
End Sem

TCL commands are as follows :

TCL statements allow you to control and manage transactions to maintain the integrity of data within SQL statements.

- **BEGIN Transaction** : opens a transaction.
- **COMMIT Transaction** : commits (Save permanently) transactions.
- **ROLLBACK Transaction** : ROLLBACK (Cancels. undo) transactions in case of any issue.

Operations performed on transactions

Commit and roll back operations are performed on transaction to manage transaction.

- **Commit** operation allow transaction to update database permanently. It will ensure the updates done by transaction will persist even if failure occurs. Commit operation leads the transaction into committed state.
- **Rollback** operation helps to maintain the atomicity property of transaction. If any failure occurs during transaction processing which leads the transaction to failed state. In this situation roll back operation cancels the changes made by that transaction in database to maintain consistency. And then the transaction will be aborted.

In general three utilities are used to manage transactions.

(1) Transaction manager

- Ensures that database proceed from one consistent state to another consistent state.
- Ensures that the transaction should not violate integrity constraints.

(2) Scheduler

- Provides a specific strategy for the execution of transaction and corresponding concurrency control.
- Avoid or resolves conflicts during concurrent data access.

(3) Recover Manager

Restore the state of database it was in before the failure of transaction.

4.6 CONCEPT OF SCHEDULE, SERIAL SCHEDULE

4.6.1 Schedule

GQ. Write note on Schedule.

(5 Marks)

- A **schedule** is a collection of several transactions that can be executed in a predefined order as a single unit.
- Schedule may combine the operations of one transaction with other transaction. These operations must be in same sequence as they occur in their individual transactions.
- Schedule is necessary in a situation when a system has several transactions which are going to be performed on same database object. If those transactions are executed in parallel, they may affect the result of the transaction.

For example

- Consider transaction T_1 and T_2 are present in the system, and both use the values of account A commonly. If one transaction is updating the values which the other transaction is accessing, then the order of these two transactions will change the result of second transaction. Hence a schedule is created to execute the transactions.
- Depending on the arrangement of these transactions, schedule can be divided into two types : **Serial schedule** and **concurrent schedule**.

4.6.2 Types of Schedule

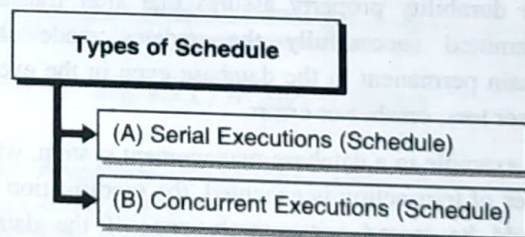


Fig. 4.6.1 : Types of schedule

4.6.2(A) Serial Executions (Schedule)

GQ. Explain Serial Execution.

(5 Marks)

- In **serial schedule** all the transactions in the schedule are executed one after another. Operations of different transactions are not interspersed with each other.
- For example, account M has \$500 and account N has \$1000 initially. Transaction T_1 transfers \$200 from account M to N and Transaction T_2 transfers \$100 from account M to N.
- T_1 and T_2 can be scheduled in two ways for execution :

Schedule 1 : Serial schedule for T₁ followed by T₂

T ₁	T ₂
Read(M);	
M=M-200;	
Write(M);	
Read(N);	
N=N+200;	
Write(N);	
	Read(M);
	M=M-100;
	Write(M);
	Read(N);
	N=N+100;
	Write(N);

After execution of both transactions M holds \$200 and N holds \$1300.

Schedule 2 : Serial schedule for T₂ followed by T₁

T ₁	T ₂
	Read(M);
	M=M-100;
	Write(M);
	Read(N);
	N=N+100;
	Write(N);
Read(M);	
M=M-200;	
Write(M);	
Read(N);	
N=N+200;	
Write(N);	

After execution of both transactions M holds \$200 and N holds \$1300.

4.6.2(B) Concurrent Executions (Schedule)

Q. Explain concurrent Execution. (5 Marks)

- Concurrent schedule** is a schedule where, several transactions are executed simultaneously. The operations of one transaction can be mixed with operations of other transaction.

Schedule 3 : Concurrent schedule for T₁ and T₂

T ₁	T ₂
Read(M);	
M=M-200;	
Write(M);	Read(M);
	M=M-100;
	Write(M);
Read(N);	
N=N+200;	
Write(N);	
	Read(N);
	N=N+100;
	Write(N);

- After execution of both transactions M holds \$200 and N holds \$1300.
- In this example concurrent schedule also provides same result as the above two serial schedule provides, But not all the concurrent schedules provide result in consistent state.

Advantages of Concurrent Execution of Transactions

Q. Explain advantages of concurrent execution of transactions. (5 Marks)

Though it is difficult to ensure the isolation while executing transactions concurrently, it has different advantages as follows.

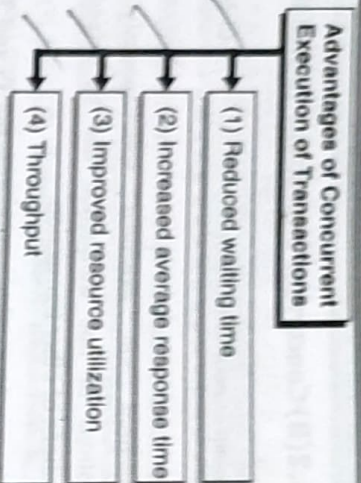


Fig. 4.6.2 : Advantages of Concurrent execution of transactions

Advantages of Concurrent Execution of Transactions

- ▶ (1) **Reduced waiting time**
System has several transactions which may be short or long. In serial execution of transaction, short transactions need to wait more when long transactions are being processed. If these transactions executed concurrently, it will reduce the waiting time.
- ▶ (2) **Increased average response time**
In concurrent execution, the multiple transactions executes concurrently by sharing various system resources, the waiting time get decreased because of which the response time automatically get increased.
- ▶ (3) **Improved resource utilization**
Number of transactions run concurrently in the system, in which if any transaction needs to wait for some operations to complete, the CPU has ability to switch to another transaction, so the CPU can be utilized in efficient way. It will no more in ideal state.
- ▶ (4) **Throughput**

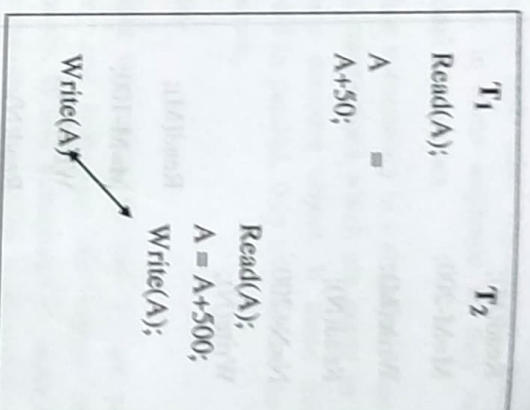
Throughput indicates the number of transaction processed in a time unit. As several transactions get executed simultaneously in the system. Due to efficient resource utilization it will produce their outputs in less amount of time. So numbers of transaction completed is more as compare to serial execution of transactions which will increase the throughput of the system. This leads to increase in system performance.

Problems Occur in Concurrent Execution of Transaction

- **Q. Write note on problems occurred in concurrent Execution of Transaction. (10 Marks)**
- The problem occurs when two transactions are dealing with same data item if one of the transactions performs write operation.

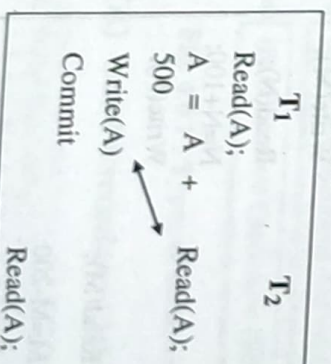
- If any transaction contains following situations then the conflicts are occurred ;

(A) Write-Write conflicts

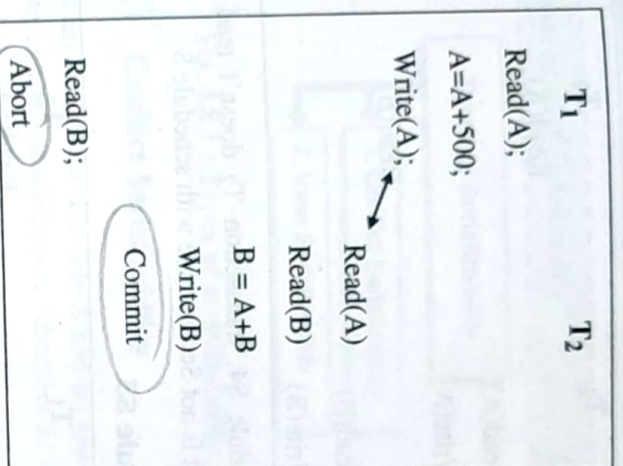


- Here both the transactions T₁ and T₂ read initial values of data item (A) [assume A=1000].
- Initially T₂ adds 500 to A and write new value 1500 of A into database.
- Now T₁ update the initial value of A(1000) to 1050 and write it in the database.
- In between these transactions, the value updated by transaction T₂ is lost.
- Thus write-write conflicts affect lost update on database.

(B) Read-write conflicts



- Transaction T₁ and T₂ reads initial value of A (1000).
- T₁ updates value of A to 1500 by adding 500 to it and writes it into database and commit.
- Now T₂ again read the value of A which is now 1500.
- Here transaction T₂ reads two different values of same data element A.
- This is read-write conflict.

(C) Write-Read conflicts

- Transaction T_1 reads value of A(1000) from database. Add 500 to it and write new value 1500 in the database.
- Transaction T_2 reads updated value of A(1500) from database, add it into B and writes value of B in the database.
- But transaction T_1 get aborted due to failure and all changes are rolled back.
- Here value read by T_2 is dirty read i.e. value updated by uncommitted transaction.

4.7 SERIALIZABILITY**Q. Explain the term : Serializability. (10 Marks)**

- If execution of all the transactions in a concurrent schedule produces same result as in serial schedule then it is called as **Serializable schedule**. **Serializability** is a property of a transaction schedule which is used to keep the data in a consistent state. It relates to the isolation property of a database transaction.

In concurrent execution of transaction, many problems may arise. Hence Serializability is must while selecting the concurrent option for execution of transactions.

Difference between Serial Schedule and Serializable Schedule

1. A schedule is said to be Serial if all the operations of transaction are executed serially without any interference from other transaction.

For example schedule 'S1' is a serial schedule if in it only one transaction executes at any given time. When a non-serial schedule produced the same result as serial schedule then it is known as Serializable schedule.

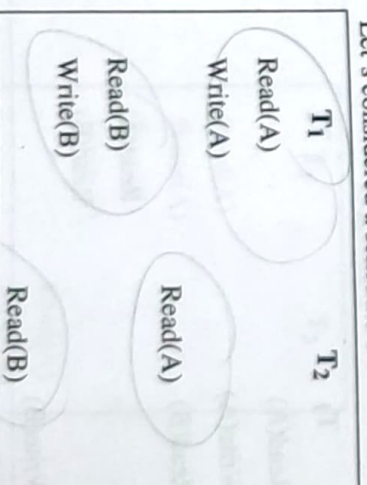
2. The main difference between serial schedule and Serializable schedule is that **Concurrency** is not allowed in serial schedule that means multiple operations cannot be performed at a time while in serializable schedule concurrency is allowed.

3. If a serial schedule includes only two transactions let's consider T_1 and T_2 , then There are only two possible orders to schedule those transactions. First is execute all the operations in transaction T_1 and then executes all the operations in transaction T_2 (i.e. T_1 followed by T_2). The second order is to execute all the operations of transaction T_2 and then execute all the operations of transaction T_1 (i.e. T_2 followed by T_1). But In Serializable Schedule, if it includes two transactions then their operations are mixed (can be executed in parallel) with each other, then there will be many possible orders in which the system can execute the individual operations of the transactions.

4. As the Serial schedule doesn't allow any concurrent execution of transactions the result of the schedule always in consistent state. But in a Serializable schedule the concurrent execution of schedule should be equal to any serial schedule so that the results of schedules are always in consistent state.

5. For example :

Let's considered a schedule S :

**Serial Schedule**

Only two serial schedules are possible for given schedule S as follow:

Serial schedule S1

T ₁	T ₂
Read(A)	
Write(A)	
Read(B)	
Write(B)	
	Read(A)
	Read(B)

Serial schedule S2

T ₁	T ₂
	Read(A)
	Read(B)
Read(A)	
Write(A)	
Read(B)	
Write(B)	

Serializable schedule

Given schedule S says that transaction T₂ reads updated value of A and reads value of B before modifying it by transaction T₁.

Let us consider 3 schedules S3, S4, and S5. We have to check whether they are Serializable with schedule S or not?

Schedule S3

T ₁	T ₂
Read(A)	
Write(A)	
Read(B)	
	Read(A)
	Read(B)
Write(B)	

In schedule S3, transaction T₂ reads updated value of A and reads value of B before modifying it by transaction T₁. This is same as schedule S. Hence we can say that the schedule S3 is Serializable with schedule S.

Schedule S4

T ₁	T ₂
Read(A)	Read(A)
Write(A)	
	Read(B)
Read(B)	
Write(B)	

In schedule S4, transaction T₂ doesn't read updated value of A. It is not Serializable with schedule S.

Schedule S5

T ₁	T ₂
	Read(A)
Read(A)	
Write(A)	
Read(B)	
Write(B)	
	Read(B)

In schedule S5, transaction T₂ doesn't read updated value of A and it reads value of B which is updated by T₁. So it is not Serializable with schedule S.

Now in brief we will see the difference between serial and Serializable Schedule.

Serial Schedule	Serializable Schedule
In serial schedule, transaction are executed one after other.	In serializable schedule, transaction are executed concurrently.
At a time, only one transaction is allowed	At a time, multiple transactions can be executed at a time.
Resultant database is always consistent	Care should be taken, to make resultant database consistent.
Processing is slow, take more time	Processing is fast.
Suitable if number of transactions are less	Suitable if number of transactions are more

4.8 TYPES OF SERIALIZABILITY

Q. Explain types of Serializability.

(10 Marks)

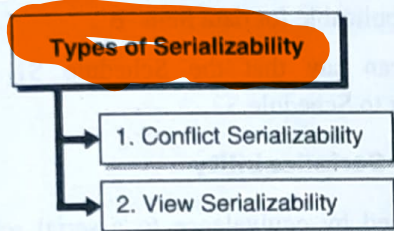


Fig. 4.8.1 : Types of serializability

4.8.1 Conflict Serializability

Let's say we have a schedule S and we can reorder the instructions in it and create 2 more schedules S1 and S2.

(i) Conflict Equivalent Schedules

- If a schedule S can be transformed into a serial schedule S' by performing series of swaps of non-conflicting operations, we can say that schedule S & S' are conflict equivalent.
- The schedule S1 and S2 are said to be conflict-equivalent if they satisfies following conditions :
 - Same set of transactions (including sequence of operations within each transaction) are involved in both schedules S1 and S2.
 - Order of pair of conflicting-operations is same in both the schedules.

(ii) Conflicting operations

Two operations belong to different transactions are said to be conflicting if they performed on same data item and at least one of them is a write operation.

Example

- Suppose if transaction T_1 has $Read_1(A)$ operation and transaction T_2 has $Write_2(A)$ operation, then this pair($Read_1(A)$, $Write_2(A)$) is called Conflicting operations pair because they belong to two different transactions on same data item A and one of them is write operation.
- Similarly, ($Write_1(A)$, $Write_2(A)$) and ($Write_1(A)$, $Read_2(A)$) pairs are also know as conflicting pairs.
- On the other hand, ($Read_1(A)$, $Write_2(B)$) pair is non-conflicting because they operate on different data items.
- Similarly, ($Write_1(A)$, $Write_2(B)$) pair is non-conflicting.

Example 4.8.1 : Check whether following schedules are conflict equivalent or not. Justify your answer. (T_1 & T_2 are transactions)

Schedule S1

T_1	T_2
Read(A)	
Write(A)	
Read(B)	
Write(B)	
	Read(A)
	Write(A)
	Read(B)
	Write(B)

Schedule S2

T_1	T_2
Read(A)	
Write(A)	
	Read(A)
	Write(A)
Read(B)	
Write(B)	
	Read(B)
	Write(B)

✓ Soln. :

Schedule 1

T_1	T_2
Read(A)	
Write(A)	
Read(B)	
Write(B)	
	Read(A)
	Write(A)
	Read(B)
	Write(B)

Schedule 2

T ₁	T ₂
Read(A)	
Write(A)	
	Read(A)
	Write(A)
Read(B)	
Write(B)	
	Read(B)
	Write(B)

• In schedule S1 which is serial transaction,

○ **Transaction T₁**

First we are performing operations read and write on A and then B.

○ **Transaction T₂**

The same read write operations are performed on A and then B.

• In Schedule S2 which is concurrent transaction,

○ **Transaction T₁**

First we are performing operations read and write on A.

○ **Transaction T₂**

The same read write operations are performed on A. Then again

○ **Transaction T₁**

Then we are performing operations read and write on B.

○ **Transaction T₂**

The same read write operations are performed on B.

- In schedule S1 (Write(A), Read(A)) and (Write(B), Read(B)) are conflicting pairs.
- As given in example, schedule S2 also follows same order of conflicting pairs as given in schedule S1.
- Here the non conflicting operations can be swapped which does not make any difference to the final result.
- Hence these two schedules are conflict equivalent.
- In other words, transaction T₂ of Schedule S1 reads the modified value of A and B.

- Now in second concurrent schedule, when transaction T₁ completes both the operations on A, then transaction T₂ perform operations on the data item A. because of which no irrelevant output can be generated. The same thing is applicable for data item 'B'.
- So we can say that the Schedule S1 is conflict equivalent to Schedule S2.

1. Conflict-Serializability

It is defined by equivalence to a serial schedule (no overlapping transactions) with the same transactions, such that both schedules have the same sets of respective chronologically ordered pairs of conflicting operations (same precedence relations of respective conflicting operations).

2. Conflict Serializable Schedule

Schedule S is conflict Serializable if it is conflict equivalent to a serial schedule. The concept of Conflict equivalence leads to the concept of conflict Serializability, we can say that the schedule S in conflict Serializable if it is conflict equivalent to a serial schedule.

Consider the following schedule : S1

T ₁	T ₂
Read ₁ (A)	
Write ₁ (A)	
	Read ₂ (A)
	Write ₂ (A)
Read ₁ (B)	
Write ₁ (B)	
	Read ₂ (B)
	Write ₂ (B)

Possible serial schedules for S1 are :

- T₁ → T₂ (i.e. all the operations of T₁ will perform first and then all the operations of T₂ will perform), and T₂ → T₁ (i.e. all the operations of T₁ will perform first and then all the operations of T₂ will perform).
- While transferring a schedule into another one following rule must be follow :

- If O_i and O_j are two operations in a transaction and $O_i < O_j$ (O_i is executed before O_j), same order will follow in new schedule as well.
- Swapping non-conflicting operations** $Read_2(A)$ and $Read_1(B)$ in S_1 , the schedule becomes,

Schedule S_1

T_1	T_2
Read ₁ (A)	
Write ₁ (A)	
Read ₁ (B)	
	Read ₂ (A)
	Write ₂ (A)
Write ₁ (B)	
	Read ₂ (B)
	Write ₂ (B)

Similarly,

- Swapping non-conflicting operations** $W_2(A)$ and $W_1(B)$ in S_1 , the schedule becomes

Schedule S_2

T_1	T_2
Read ₁ (A)	
Write ₁ (A)	
Read ₁ (B)	
Write ₁ (B)	
	Read ₂ (A)
	Write ₂ (A)
	Read ₂ (B)
	Write ₂ (B)

- S_2 is a serial schedule in which all operations of T_1 are performed before starting any operation of T_2 . Since the S_1 has been transformed into a serial schedule S_2 by swapping non-conflicting operations of S_1 , So S_1 is **conflict serializable**.

4.8.2 View Serializability

- View-serializability** of a schedule is defined by equivalence to a serial schedule (no overlapping transactions) with the same transactions, such that

respective transactions in the two schedules read and write the same data values ("view" the same data values).

- The schedules S_1 and S_2 are view equivalent if they follows following conditions :
 - In schedule S_1 , for data item A if read operation is performed by transaction T_i , then in schedule S_2 , transaction T_j must perform read operation on data item A.
 - If in schedule S_1 , transaction T_i reads the data written by T_j , then the schedule S_2 , must have the read operation in transaction T_i , who reads the data written by T_j .
 - If transaction T_i writes the last value of Data item A in schedule S_1 then, in schedule S_2 last write operation on data item A must be performed by Transaction T_i .
- First two conditions ensure that no conflict occurs during reading the values of data item 'A' in both the schedules. The last condition ensures that the final result of both the schedules is same.
- The concept of view equivalence leads to view serializability. The schedule S_1 is view serializable if it is view equivalent to a serial schedule.

T_1	T_2	T_3
read(A)		
	write(A)	
write(A)		
		write(A)

Schedule S

- Here the Schedule S is view equivalent to the serial schedule $\langle T_1, T_2, T_3 \rangle$, because the read(A) reads the initial value of A in both schedules and T_3 performs write(A) in both schedules.
- Every view Serializable schedule is not a conflict Serializable schedule, but every conflict Serializable schedule is known as view Serializable schedule.

4.9 RECOVERABILITY : RECOVERABLE AND NON-RECOVERABLE SCHEDULES, CASCADED ABORTS

4.9.1 Recoverable Schedule

Q. Explain Recoverability.

(10 Marks)

If any schedule S is getting correct initial value of the data item after recovering it from failure is called as **recoverable schedule**. The database remains consistence in this schedule.

Consider two transactions T_1 and T_2 . Recoverable schedule has some rules as follows :

- In transaction schedule S , for each pair of transaction T_1 and T_2 , is schedule such that T_2 can read the data item which is previously written by T_1 .
- Commit operation of T_1 must occur before the commit operation of T_2 .
- If the rules are not followed then it is difficult to recover the schedule from inconsistent state.

Consider an example

T_1	T_2
Read(X)	
Write(X)	
	Read(X)
Read(Y)	

- Suppose after T_2 Read(x) operation, it commits. Then somehow T_1 fails. So the transaction T_2 must be aborted so as to ensure atomicity. However, since T_2 is committed and can't be aborted. Hence a situation arrives where it is impossible to recover. And hence it is necessary that T_1 commits before T_2 .
- **Recoverable schedules are desirable** to prevent the system from going into inconsistent state permanently after failure occurs in the system.

4.9.2 Non Recoverable Schedule

- If any schedule S is not getting correct value of the data item after recovering it from failure is called as non-recoverable schedule.
- For example, the schedule S_1 contains two transactions T_1 and T_2 as follows:
- Value of M is initially 100.

T_1	T_2
Read(M);	
$M=M+200$;	
Write(M);	
	Read(M);

T_1	T_2
Abort;	$M=M+100$;
	Write(M);
	Commit;

- As shown in example Transaction T_1 modifies the value of M to 300, then transaction T_2 modifies value of M to 400 and T_2 committed its transaction. After that, the transaction T_1 aborted due to some failure in the system. Recovering it from failure leads to reassign the value of M to 400 (instead of 300) as it is committed value by T_2 transaction. This type of scheduling is not recoverable.

4.9.3 Cascading Schedule

- Consider an example where three transactions are scheduled T_1 , T_2 and T_3 :
- In which T_2 reads the value of M which is written by T_1 and T_3 reads the value which is written by T_2 . Such schedule of transactions is known as cascading schedule.

T_1	T_2	T_3
Read(M);		
Read(N);		
$M=5000$;		
Write(M);		
Read(N);		
	Read(M);	
	$M=10000$;	
	Write(M);	
		Read(M);

- If a single transaction fails then series of transactions needs to be rollback. This rollback operation is called as Cascading rollback.

4.9.4 Cascaded Aborts

- When abort of T_i transactions leads to abort T_j transaction, this is called as **Cascaded aborts**.
- If any failure occurs in the T_1 transaction then T_1 needs to abort (abort operation simply rollbacks the performed operations so that it should recover from

failure). As the T_2 depends on T_1 transaction, both needs to be aborted and rollback to recover from failure. Also T_3 needs to abort and rollback because it is depend on the value provided by T_2 . This leads to cascading rollback/abort, this leads to unnecessary undoing the work.

4.9.5 Cascadeless Schedule

- It is the types of recoverable schedule where, cascading rollback is not allowed. So that throughput of the system will increase.
- Consider an example where three transactions are scheduled T_1 , T_2 and T_3
- In which T_2 reads the value of M which is written by T_1 and T_3 reads the value which is written by T_2 . If any failure occurs in the T_1 transaction, as the T_2 is depend on T_1 transaction, both needs to be rollback to recover from failure also T_3 needs to rollback because it is depend on the value provided by T_2 . This leads to cascading rollback, this leads to unnecessary undoing the work.

T_1	T_2	T_3
Read(M);		
Read(N);		
M=5000;		
Write(M);		
Read(N);		
	Read(M);	
	M=10000;	
	Write(M);	
		Read(M);

- So it is necessary to avoid cascading rollbacks in schedules to increase throughput of the system.
- The Cascadeless schedule is a type of schedule where each transaction pair T_i and T_j is scheduling such that :
 - T_j can read the data written by T_i .
 - Commit operation of T_i must be performed before the commit operation of T_j .

- If any failure occurs then no cascading rollback operations are needed. Only some parts of transactions need to be rollback.

4.9.6 Strict Schedule

It is the types of schedule in which no other transaction will perform Read(M) or Write(M) until the last transaction which performed on data item M is committed or aborted. Strict schedule is most restricted than Cascadeless schedule.

T_1	T_2
Read(M);	
M=5000;	
Write(M);	
Commit;	
	Read(M);

4.10 CONCURRENCY CONTROL : NEED

Q. Explain concurrency control in distributed database.

- Concurrency control is an important concept of database management systems which is used to address conflicts with the concurrent access or modification of data that can occur with a multi-user system.
- This system helps to coordinate the multiple transactions executing simultaneously by preserving data integrity.
- Concurrency control ensures that correct results for concurrent operations are generated, while getting those results as quickly as possible.

4.10. Need of Concurrency Control

Concurrency control is important in DBMS to achieve the simultaneous execution of the transactions. This can be done by executing few instructions of one transaction then next and the process is going on till the executions of transaction get completed.

Several problems can occur when concurrent transactions execute in an uncontrolled manner.

(1) The Lost Update Problem

Lost update problem occurs when two or more transactions are modifying the value of same data items simultaneously. In this the final value of the data item can be decided by the transaction which commits at last among those transactions.

Example

- The initial value of both data items P and Q is 1000. The transaction T_1 transfers 50 from P to Q. Transaction T_2 withdraw 10% from amount of P.
- If T_1 and T_2 get executed serially then final values of P and Q are as follows :

If T_1 then T_2	If T_2 then T_1
Final values are P = 855 Q = 1050	Final values are P = 850 Q = 1050

- In above example T_1 lost its update due to the action of T_2 .

(2) The Temporary Update (or Dirty Read) Problem

The temporary update problem is known as Dirty Read Problem. It occurs when one transaction permitted to read the data item that is modified by an uncommitted transaction. The problem occurs when uncommitted transaction decides to rollback.

Example

If transaction T_1 modify data item P and transaction T_2 is allowed to read the value of P as modified by T_1 before T_1 commits. At this time dirty read problem occurs. The read operation which performed by the T_2 is called as dirty read operation.

(3) The Incorrect Summary Problem

When one transaction performs aggregate function on several data items, and other transaction are updating those data. At that time incorrect summary problems occurs.

Example

T_1 is calculating total balances of account x (100), account y (50), and account z (25). Meantime, T_2 has transferred 10 from x to z, so T_1 now has wrong result.

4.11 LOCKING METHODS

Q. Explain different types of Locking Methods.

(10 Marks)

- A lock is a mechanism to control concurrent access to a data item.
- When multiple transactions are accessing some data item at a time, it may lead to generate some irrelevant data generation. Hence the data item should be isolated so that it should be accessible in a mutually exclusive manner that is when one transaction is accessing the data item, no other transaction should be able to access that item.
- It can implement by a way in which a transaction can access the item only if it currently holds lock on the data item.

There are two types of locks.

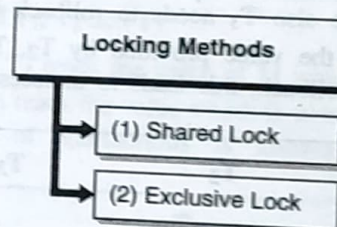


Fig. 4.11.1 : Locking Methods

(1) Shared Lock

- The shared lock is also known as read lock. Shared lock is acquired on the transaction when concurrent transactions granted the permission for read access on the data.
- A shared lock is issued when a transaction wants to read data from the database and no exclusive (write) lock is held on that data item.

(2) Exclusive Lock

- This lock is also known as write lock. This lock is issued when a transaction needs to write or update a data item and there is no lock currently held on that item. This lock is placed on data item when a write or update operation is perform. Only one exclusive lock is placed on data at a time.
- In this system, every transaction will request for appropriate lock (shared or exclusive) depending upon the type of operation which it want to perform on the database.

- There is a utility known as concurrency control manager which grants the locks to transactions. The transactions requests for a lock to this concurrency control manager.
- The locking mechanism allows reading the data items simultaneously for various transactions but allows only one transaction to perform write operation on the data item at a time.

4.12 DIFFERENT CONCURRENCY CONTROL PROTOCOLS

To ensure transaction properties such as atomicity, isolation and Serializability of concurrent transactions, concurrency control protocols are used. Concurrency control protocols can be broadly divided as follows

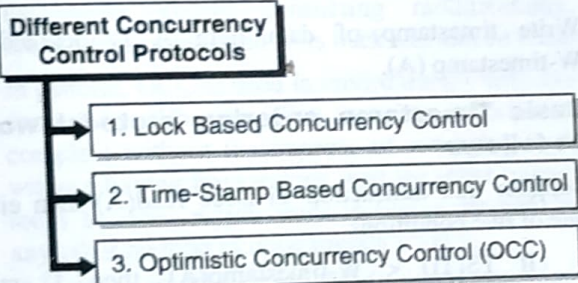


Fig. 4.12.1 : Different Concurrency control protocols

4.12.1 Lock Based Concurrency Control

(A) Two-Phase Locking (2PL)

Q. Explain different two phase locking protocols in transaction management. (5 Marks)

- In this scheme, each transaction makes lock and unlock request in 2 phases :
 1. **A Growing Phase(or An Expanding Phase or First Phase) :** In this phase, new locks on the desired data item can be acquired but none can be released.
 2. **A Shrinking Phase (or Second Phase) :** In this phase, existing locks can be released but no new locks can be acquired.
- In the beginning, the transaction is in growing phase in which it acquire lock as per requirement. After completing the work, the transaction releases the locks and enters into shrinking phase.
- The transaction cannot request for new locks after releasing the locks.
- The point in the schedule where the transactions acquire its final lock is called as **lock point**. This point is the end of growing phase. The transactions can be ordered as per their lock points. This sequence of transactions is the serializability ordering for the transactions. This protocol assures serializability.

```

T
Lock-X(A);
Read(A);
A = A + 500;
Write(A);
Lock-X(B);
Read(B);
B = B - 100;
Write(B);
Unlock(A);
Unlock(B);
  
```

- In Serializability the main issue is regarding write operation. The parallel write operations may create inconsistency in the database. The parallel read operations do not create any problem.
- In serial schedule, as the transactions are executed one after other, there is no risk of parallel write operations which may lead to inconsistency.
- The database transactions can be controlled in 2-Phase locking mechanism by applying the exclusive lock. The exclusive (write) lock is released in shrinking phase. Unless the exclusive lock of a data item is released by the transaction, other transaction can acquire exclusive lock on that data item. This helps to maintain the serializability. It is not sure that Two-phase locking is free from deadlocks.
- In two-phase locking, there is possibility of cascading roll-back. This can be avoided by using **strict two-phase locking** in which a transaction must hold all its exclusive locks till it commits/aborts.
- **Rigorous two-phase locking** is more stricter. In it all the locks are held till commit/abort. Here transactions can be serialized in the order in which they commit.

(B) Strict Two-Phase Locking

- In Strict two phase locking protocol the first step of acquiring all the locks is same as first phase of two phase locking protocol.
- But the difference in Strict two phase locking and two phase locking is that Strict two phase locking protocol does not release the lock after using it.
- It holds all the locks till it does not reach to commit point. When it reached to the commit point it releases all locks at a time.
- Cascading rollback can be prevented by using the strict phase locking protocol. This protocol is used to make Cascadeless schedules.

4.12.2 Time-Stamp Based Concurrency Control

Q. Explain the Time Stamp based concurrency control. (10 Marks)

- To identify the relative starting time of a transaction, DBMS creates a unique identifier called as **Timestamp**.
- In general the values of Timestamp are assigned in the order in which the transactions are submitted to the system. Hence Timestamp is considered as a method of concurrency control in which transaction timestamp is assigned. To all the transactions.
- Timestamps must have two properties namely
- The timestamp should be unique.
- The timestamp values always increase.
- In timestamp method, as soon as a transaction is created, the timestamp is assigned to it. Timestamp for transaction is generated by using the system time. Timestamp is used to identify transaction uniquely.
- The older transaction has first priority to execute.
- The timestamp of transaction T_i is denoted as $TS(T_i)$. Concurrency control techniques which are based on timestamp ordering don't use locks; so deadlocks cannot occur in the system.

(A) Timestamp Generation

- Timestamps can be generated in several ways.
- One way is to assign numbers to each transaction when it is created. For this, counter is used. The transaction timestamps are numbered as 1, 2, 3, ... n in this scheme. The counter is reset to zero when no transactions are executing.

- System's current date/time value can be used to implement timestamps. It is another way to generate timestamp value.

(B) Timestamp Ordering Algorithm

- The timestamp-ordering protocol used to order the conflict read and write operation pairs in such a way that they maintain Serializability.
- Timestamp ordering protocol ensures Serializability of conflict read/write operation pairs. This protocol ensures Serializability by assuring that the conflicting pair of tasks should be executed according to the timestamp values of the transactions.
- The timestamp of transaction T_i is denoted as $TS(T_i)$.
- Read timestamp of data item A is denoted by R-timestamp (A).
- Write timestamp of data item A is denoted by W-timestamp (A).

Basic Timestamp ordering protocol works as follows

- Suppose that transaction T_i issue read(A) then either one of two conditions
 - If $TS(T_i) < W\text{-timestamp}(A)$, then T_i reads overwritten value of A. So, the read operation on data item A is rejected, and transaction T_i is rolled back.
 - If $TS(T_i) \geq W\text{-timestamp}(A)$, then the read operation on data item A is executed, and $R\text{-timestamp}(A)$ is set to the maximum of $R\text{-timestamp}(A)$ and $TS(T_i)$ i.e. $R\text{-timestamp}(A) = \max(R\text{-Timestamp}(A), TS(T_i))$.
- Suppose that transaction T_i issues Write(A)
 - If $TS(T_i) < R\text{-timestamp}(A)$, then the system rejects the write operation and T_i is rolled back.
 - If $TS(T_i) < W\text{-timestamp}(A)$, which means T_i is attempting to write an obsolete value of A. So, the write operation is rejected and T_i is rolled back.
 - Otherwise, the write (A)m operation is executed.

Thomas' Write Rule

This rule states

- Suppose that transaction T_i issues Write(A).
If $TS(T_i) < W\text{-timestamp}(A)$ and $TS(T_i) < R\text{-timestamp}(A)$, then instead of rejecting the Write(A) operation it can be ignored and obsolete writes are deleted.

4.12.3 Optimistic Concurrency Control (OCC)

Q. Explain the Optimistic Concurrency Control.

(10 Marks)

- It is a concurrency control method applied to transactional systems such as relational database management systems and software transactional memory.
- OCC assumes that more than one transactions can be frequently completed without interfering with each other.
- At the time of running, data resources are used by transactions without acquiring locks on those resources. Before committing, each transaction verifies that no other transaction has modified the data it has read. If the check shows conflicting modifications, the committing transaction rolls back and can be restarted.
- In general, OCC is used in environments with low data contention. When conflicts are rare, transactions can complete without the expense of managing locks and without having transactions wait for other transactions' locks to clear, which leads to higher throughput than any other method of concurrency control.
- However, if the process of contention for data resources is frequent, the cost of repeatedly restarting transactions may affect performance significantly; it is commonly thought that in such situations, other concurrency control methods have better performance.
- However, locking-based ("pessimistic") methods also can generate poor performance because locking can significantly limit effective concurrency even when deadlocks are avoided.

More specifically, OCC transactions involve these phases :

- Begin** : Record a timestamp marking the transaction's beginning.
- Modify** : Read database values, and tentatively write changes.
- Validate** : Check whether other transactions have modified data that this transaction has used (read or written). This includes transactions that completed after this transaction's start time, and optionally, transactions that are still active at validation time.
- Commit/Rollback** : If there is no conflict, make all changes take effect. If there is a conflict, resolve it, typically by aborting the transaction, although other resolution schemes are possible. Care must be taken to avoid a TOCTTOU (Time-of-Check Time-of-Use) bug, particularly if this phase and the previous one are not performed as a single atomic operation.

4.13 DEADLOCK

Q. When deadlock occurs? How to prevent them? How to recover if dead lock takes place? (10 Marks)

- A system is said to be in a state of Deadlock, if every transaction in the schedule is waiting for another transaction in the schedule to release the lock of some data item. Consider transactions from T_0 to T_n . In this case transaction T_1 is waiting for T_2 , T_2 is waiting for T_3 and so on. And at the last the transaction T_n is waiting for transaction T_0 in cyclic manner.
- The transactions present in deadlock are either rollback or restarted. A system with a deadlock indicates bad behavior of the system. The rollback of the transaction may be partial. That is a transaction may rolled back to the point where it obtained a lock whose release resolves the deadlock.
- It's not good to have any transaction that causes deadlock. To handle the deadlock situation, we have two options one is to avoid a system to enter in deadlock by Deadlock Prevention method or after deadlock try to recover it by deadlock detection and deadlock recovery method.

So to handle deadlocks following methods are used :

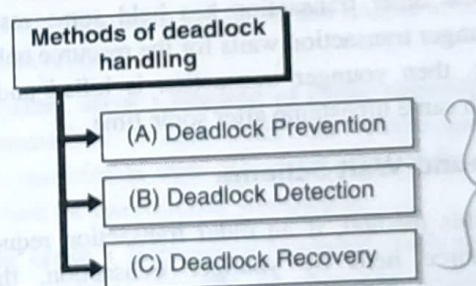


Fig. 4.13.1 : Methods of deadlock handling

4.13.1 Deadlock Prevention

- DBMS regularly checks all the transaction operations which are going to execute, to prevent any deadlock situation in the system. The checking of all the operations is done to analyze whether the execution of them leads the system in deadlock or not. If any operation of a transaction leads the system in deadlock state then that transaction will not allow for execution.
- There are different methods to prevent deadlock. All methods use the timestamp of transaction. The timestamp of any transaction T_i is denoted as $TS(T_i)$.

Wait-Die Scheme

- In wait-die scheme if the older transaction requests resources held by younger one then DBMS allows T_i to wait for needed resources but if the younger transaction requests resources held by older one then DBMS kills T_j and restarted it later.
- Suppose if a transaction T_i requests for the resources which are already locked by other transaction T_j . Then in this scheme the DBMS checks for the timestamp of both the transaction T_i and T_j and performs following action.
- If $TS(T_i) < TS(T_j)$ (means T_i is the older transaction than T_j) and T_i requests resources which are held by T_j , then DBMS allows T_i to wait until resource is available for execution. That means if a younger transaction has locked some resource and older transaction needs those resources so older transaction is waiting for younger transaction to complete, then older transaction is allowed wait for it till it is available.
- $TS(T_i) < TS(T_j)$ (means T_i is the older transaction than T_j) and T_j requests resources which are held by T_i , then DBMS killed the T_j and restarted it latter with random delay but with the same timestamp. That means if the older transaction has held some resource and younger transaction waits for the resource held by older one, then younger transaction is killed and restarted with same timestamp after some time.

Wound-Wait Scheme

- In this method, if an older transaction requests for a resource held by younger transaction, then older transaction forces younger transaction to kill the transaction and release that resource. The younger transaction is restarted later with same timestamp.
- But if the younger transaction is requesting a resource which is held by older one, then DBMS allows younger transaction to wait until the older releases that resource.

4.13.2 Deadlock Detection

- Detecting deadlock situation in advance is always good instead of aborting a transaction. A deadlock avoidance mechanism is used to detect deadlock situation in advance. An algorithm that examines the state of the system is called periodically to check the occurrence of deadlock.

- In deadlock avoidance "wait-for graph" method is used.

Wait-for Graph

"Wait-for-graph" scheme uses a graph (set of edges and vertices (nodes)) to detect deadlock in the system.

- A node is created when a transaction enters into system.
- When a transaction T_i requests for a lock on resource X which is held by some other transaction T_j , a directed edge is created from T_i to T_j .
- This edge is removed when T_j releases resource X , and T_i locks that resource.
- Waits for graph indicates which transaction is waiting for another transaction to complete.
- A graph with cycle indicates the occurrence of deadlock in the system.
- The wait for graph for Fig. 4.13.2 has following situation :

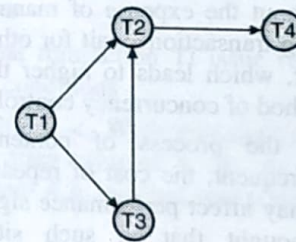


Fig. 4.13.2

- Transaction T_1 is waiting for transaction T_2 and T_3 .
- Transaction T_3 is waiting for transaction T_2 .
- Transaction T_2 is waiting for transaction T_4 .
- Here the graph has no cycle; hence it is not in deadlock state.
- Consider the transaction T_4 is requesting an item held by T_3 then a directed edge from T_4 to T_3 will be drawn.
- Now the graph has a cycle - $T_2 \rightarrow T_4 \rightarrow T_3 \rightarrow T_2$
- Here the transactions T_2 , T_3 AND T_4 are in deadlock state.

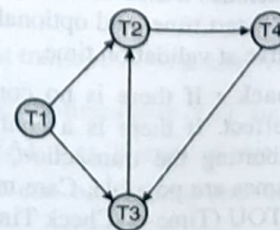


Fig. 4.13.3

4.13.3 Deadlock Recovery

When deadlock is detected by the system, the only one solution is recovery of deadlock. The best way of recovery is rollback operation.

In the recovery process, three actions are taken

1. Selection of victim

The first step is to detect that which transactions should be rolled back to recover the system. It is necessary to rollback the transaction which has minimum cost. The minimum cost depends upon various factors :

- The data items used by the transaction
- Time duration for the transaction to complete its assigned task.
- Number of total transactions need to be rolled back.

2. Rollback

After finalizing the transaction to be rolled back, we have to decide upto which previous point it should be rolled back.

Rollback operation has two options :

Total rollback

Abort the whole transaction and restart it.

Partial rollback

- Rollback transaction as per necessity. But this format requires the system to store state of all the running transactions. The order of lock requests/grants and updates performed by the transaction should be recorded.
- The deadlock detection mechanism should decide which locks the selected transaction need to release to resolve the deadlock. The selected transaction should be released up to the point where the first lock is obtained. All the operations after that point should be undone.
- The transaction must be capable of resuming execution after the partial rollback.

3. Starvation

- In deadlock handling mechanism the selection of transaction to rollback(victim) depends upon the cost fact. Sometimes it is possible that same transaction may

be selected multiple times in multiple deadlock situations. In such case it will be difficult for the victim transaction to complete its execution. It is called as **starvation**.

- Hence it is necessary that, a transaction should not be selected as victim many number of times. To achieve this, the number of rollbacks of every transaction should be maintained in the cost factor.

4.14 RECOVERY SYSTEM

- Failure is a routine situation for any system. There are various causes of failure.
- Hence failure can be classified as follows.

4.14.1 Failure Classification

GQ. Write note on Failure Classification. (5 Marks)

1. **Transaction failure** : A transaction may fail mainly because of two reasons.
 - (a) **Logical error** : There are some internal conditions like invalid input, data not found, overflow or resource limit exceeded, because of which the system get failed and unable to continue its execution.
 - (b) **System error** : Because of cyclic dependency of transactions on each other , the system may enter in undesirable state like deadlock which leads to break the execution of transaction.
2. **System crash** : Because of hardware malfunction or bug in database software or operating system, the system may halt. The loss of contents of volatile storage is possible is this crash.
3. **Disk failure** : Failure during data transfer or head crash may cause the loss of data in a disk.

Whatever is the reason of failure, it leads to data loss and inconsistency in the database system, and hence recovery is very important.

4.14.2 Storage Structure

GQ. Explain Storage structure in recovery system. (5 Marks)

Usually, the storage structure is divided into two categories :

4.15 LOG BASED RECOVERY

Q. Explain log based recovery scheme. (8 Marks)

Log

- **Log** is a data structure which is used to store all modifications performed on the database.
- Log contains sequence of log records which keeps track of update activities performed on the database.
- Log is an essential part of recovery system to handle the failures.
- So it is necessary to store the logs in storage where data should not be loss due to any reason. So the log records are stored on stable storage.

Log-based Recovery

- If any failure occurs in the system, with the help of logs, system can be recovered. This type of recovery is called as **log based recovery**.
- The Log-based recovery works in following manner.
- The log file is placed on a stable storage media.
- When a transaction enters the system and starts execution, it writes a log about it.

<T_n, Start>

- When the transaction updates an item X, it write logs as follows -

<T_n, X, V₁, V₂>

- It reads that the value of X is changed by T_n from V₁ to V₂.
- When the transaction finishes, it logs -

<T_n, commit>

- There are two techniques for using log to achieve recovery and ensure atomicity in case of failures.
- Deferred database modifications
 - This technique delay the database modification until transaction committed.
 - It allows storing all the modification done by transaction in the database, when that transaction committed successfully.

1. **Volatile storage** - As the name suggest, it is not possible for volatile storage to survive system crashes.

Volatile storage devices are located near to the CPU; usually they are embedded onto the chipset itself.

The examples of volatile storage are main memory and cache memory.

Their speed is good but can store only a small amount of information.

2. **Non-volatile storage** - These memories are basically built to survive system crashes.

Their data storage capacity is huge, but accessibility is slow.

Examples of non-volatile storage may are hard-disks, magnetic tapes, flash memory, and non-volatile (battery backed up) RAM.

4.14.3 Recovery and Atomicity

Q. Write note on Recovery and Atomicity. (5 Marks)

Whenever a system gets crashed, it may have number of transactions being executed and several files opened for them to update the data items.

Transactions contain several operations, which are atomic in nature. But according DBMS' ACID properties, it is necessary that atomicity of transactions as a whole must be maintained, that means either all the operations should be executed or none of them.

After recovery from a crash, a DBMS should maintain the following :

- It should verify the states of all the transactions being executed.
- It is possible that a transaction may be in the middle of any operation; in this case the DBMS should ensure the atomicity of the transaction.
- It should check whether it is possible to complete the transaction now or it needs to be rolled back.
- No any transaction should be allowed to leave the DBMS in an inconsistent state.
- For DBMS recovery there are two types of techniques which maintain the atomicity of transaction.

(a) Log based Recovery (b) Shadow paging

Immediate database modification

- Immediate database modification technique allows storing each modification done by transaction in to database as soon as it is done.
- In this technique database modification operation doesn't wait for commit operation to be performed by transaction.
- The system allows modifying the database when transaction processing is performed and not committed yet. Database modification performed by active transaction is known as uncommitted modification.

4.16 CHECKPOINTS

Q. Write note on checkpoints.

(5 Marks)

- In DBMS storing large number of logs may fill out all the memory space available in the system. The size of log may increase tremendously as time goes which makes difficult to handle it. With the checkpoint mechanism we can remove all previous logs from the system and store them on a storage disc permanently. Using Checkpoint a point can be declared before which the DBMS was in consistent state and all the transactions were committed.
- In immediate database modification recovery technique, some transactions are requires to be redone and some requires to be undone by searching the entire log. So the searching entire log is time consuming process. To overcome this problem check points are used.
- Consider a system with concurrent transactions crashes and recovers, it behaves in the following manner.

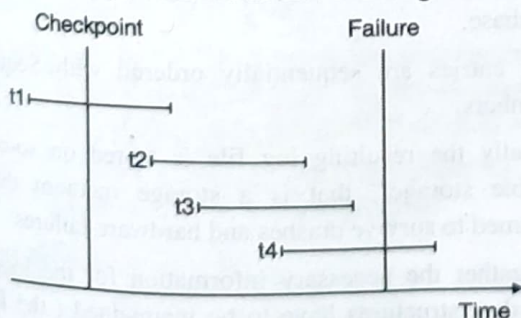


Fig. 4.16.1 : Checkpoints

- During execution along with keeping log using immediate database modification and deferred database modification technique, checkpoints are periodically performed by the system, which perform following sequence of steps :

- All the logs reside currently on main memory are stored permanently on the disk.
- All buffer blocks which are modified currently are written to the disks permanently.
- A log record <checkpoint> also written onto the disk permanently.
- While processing the checkpoint transactions are not allowed.
- One advantage of maintaining checkpoint is that, if in set of transactions $\langle T_i, \dots, T_n \rangle$ the most recent checkpoint is maintained at transaction T_j where, $i < j$ and $j < n$. i.e. T_j is in between T_i and T_n .
- Then for recovery purpose keeping log records of transactions which performed in between T_j to T_n is preferable. And deleting the unnecessary log records which are done before the checkpoint T_j is allowed.
- This means that the transaction recovery can start from the checkpoint log record and not from first record in the log. So it reduces the time.

4.17 SHADOW PAGING

Q. Write note on Shadow Paging.

(5 Marks)

- The **shadow paging** does not require the use of a log in a single-user environment. In this scheme database is considered to be made up of a number of fixed-size disk pages or blocks for recovery purposes.
- A directory with n entries is created. In it the i^{th} entry points to the i^{th} database page on disk. This directory is kept in main memory. It records references of all reads or writes to database pages. When execution of transaction started we copy the current directory whose entries points to the most recent database pages on disk into a shadow directory.
- The current directory is used by the transaction and the shadow directory is saved on disk.
- While executing the transaction, no changes are made in shadow directory. If a 'write' operation is performed, then new copy of modified database page is created. Both new and old copies are kept. That means for pages which are updated during the transaction, old and

Unit
IV
End Sem



new, both versions are kept. The shadow directory refers the old version while the current directory refers the new version.

- In case of failure, for recovery purpose the modified database pages are freed and current directory is discarded.
- The shadow directory provides the state of the database before transaction execution. The state is recovered by restoring the shadow directory. In this way the database returned to its state prior to the transaction that was executing when the crash occurred and any modified pages are discarded.

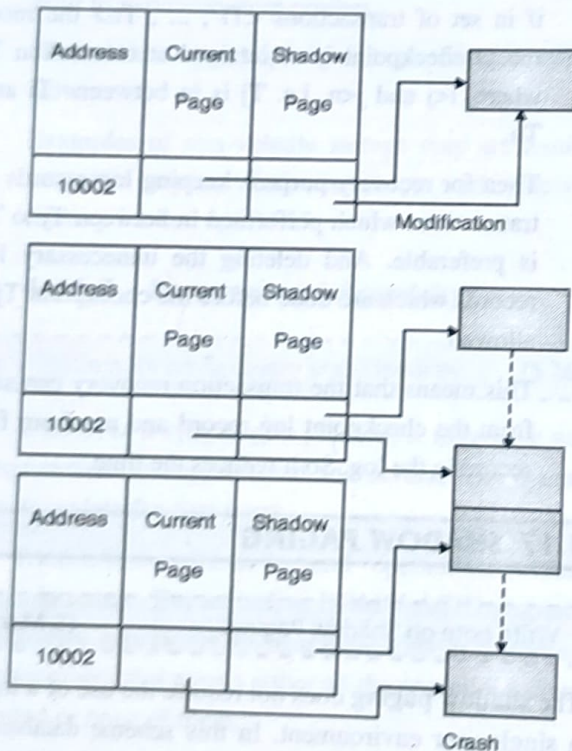


Fig. 4.17.1 : Shadow Paging

- This technique can be categorized as a NO-UNDO/NO-REDO technique for recovery of database because for recovery, neither undoing nor redoing data items is performed.
- In case of multiuser system where transactions are performed concurrently, logs and checkpoints are used in the shadow paging technique.
- One of disadvantage of this technique is that the database pages which are updated, change location on disk. Because of this, it is difficult to keep related database pages close together on disk. It requires complex storage management strategies to manage those pages.

4.18 ARIES ALGORITHM

- GQ.** Describe the three phases of the ARIES recovery method.
- GQ.** What are the roles of the analysis, Redo and Undo phases in ARIES ? Explain with suitable example.
- GQ.** Describe the ARIES recovery algorithm with example.

- In computer science, Algorithms for Recovery and Isolation Exploiting Semantics, or ARIES is a recovery algorithm designed to work with a no-force, steal database approach.
- Dr. C. Mohan is the primary inventor of the ARIES family of algorithms.

There are three main principles lie behind ARIES :

- Write-ahead logging :** Any change to an object is first recorded in the log, and the log must be written to stable storage before changes to the object are written to disk.
- Repeating history during Redo :** On restart after a crash, ARIES retraces the actions of a database before the crash and brings the system back to the exact state that it was in before the crash. Then it undoes the transactions still active at crash time.
- Logging changes during Undo :** Changes made to the database while undoing transactions are logged to ensure such an action isn't repeated in the event of repeated restarts.

Logging

- For the ARIES algorithm to work a number of log records have to be created during the operation of the database.
- Log entries are sequentially ordered with Sequence Numbers.
- Usually the resulting log file is stored on so-called "stable storage", that is a storage medium that is assumed to survive crashes and hardware failures.
- To gather the necessary information for the logging, two data structures have to be maintained : the **Dirty Page Table (DPT)** and the **Transaction Table (TT)**.
- The dirty page table keeps record of all the pages that have been modified and not yet written back to disc and the first Sequence Number that caused that page to become dirty.

- The transaction table contains all transactions that are currently running and the Sequence Number of the last log entry they caused.
- We create log records of the form (Sequence Number, Transaction ID, Page ID, Redo, Undo, Previous Sequence Number).
- The Redo and Undo fields keep information about the changes this log record saves and how to undo them.
- The Previous Sequence Number is a reference to the previous log record that was created for this transaction.
- In the case of an aborted transaction, it's possible to traverse the log file in reverse order using the Previous Sequence Numbers, undoing all actions taken within the specific transaction.
- Every transaction implicitly begins with the first "Update" type of entry for the given TransactionID, and is committed with "End Of Log" entry for the transaction.
- During a recovery or while undoing the actions of an aborted transaction a special kind of log record is written, the Compensation Log Record (CLR), to record that the action has already been undone.
- CLRs are of the form (Sequence Number, Transaction ID, Page ID, Redo, Previous Sequence Number, Next Undo Sequence Number).
- The Redo field contains application of Undo field of reverted action, and the Undo field is omitted because CLR is never reverted.

Recovery

- The recovery works in three phases.
- The first phase, Analysis, computes all the necessary information from the log file.
- The Redo phase restores the database to the exact state at the crash, including all the changes of uncommitted transactions that were running at that point in time.
- The Undo phase then undoes all uncommitted changes, leaving the database in a consistent state.

Analysis

- During the Analysis phase we restore the DPT and the TT as they were at the time of the crash.
- We run through the log file (from the beginning or the last checkpoint) and add all transactions for which we encounter Begin Transaction entries to the TT.
- Whenever an End Log entry is found, the corresponding transaction is removed.

- The last Sequence Number for each transaction is of course also maintained.
- During the same run we also fill the dirty page table by adding a new entry whenever we encounter a page that is modified and not yet in the DPT.
- This however only computes a superset of all dirty pages at the time of the crash, since we don't check the actual database file whether the page was written back to the storage.

Redo

- From the DPT we can compute the minimal Sequence Number of a dirty page.
- From there we have to start redoing the actions until the crash, in case they weren't persisted already.
- Running through the log file, we check for each entry, whether the modified page P on the entry exists in the DPT table.
- If it doesn't, then we do not have to worry about redoing this entry since the data persists on the disk.
- If page P exists in the DPT table, then we see whether the Sequence Number in the DPT is smaller than the Sequence Number of the log record (i.e. whether the change in the log is newer than the last version that was persisted).
- If it isn't, then we don't redo the entry since the change is already there.
- If it is, we fetch the page from the database storage and check the Sequence Number stored on the page to the Sequence Number on the log record.
- If the former is smaller than the latter, the page needs to be written to the disk.
- That check is necessary because the recovered DPT is only a conservative superset of the pages that really need changes to be reapplied.
- Lastly, when all the above checks are finished and failed, we reapply the redo action and store the new Sequence Number on the page.
- It is also important for recovery from a crash during the Redo phase, as the redo isn't applied twice to the same page.
- For example, if a change recorded in the log pertains to page P1 that is not in the DPT, and then this change is already on disk and does not need to be reapplied.
- Or, if a change recorded in the log (with LSN = N where LSN is : The log sequence number) pertains to page P1 and the DPT include an entry for P1 with LSN greater than N, then the change is already present.



- If neither of these two conditions hold, page P1 is read from disk and the LSN stored on that page, LSN (P1), is compared with N . If $N < \text{LSN}(P1)$, then the change has been applied and the page does not need to be rewritten to disk.

Undo

- After the Redo phase the database reflects the exact state at the crash.
- However the changes of uncommitted transactions have to be undone to restore the database to a consistent state.
- For that we run backwards through the log for each transaction in the TT (those runs can of course be combined into one) using the Previous Sequence Number fields in the records.
- For each record we undo the changes (using the information in the Undo field) and write a compensation log record to the log file.
- If we encounter a Begin Transaction record we write an End Log record for that transaction.
- The compensation log records make it possible to recover during a crash that occurs during the recovery phase.
- That isn't as uncommon as one might think, as it is possible for the recovery phase to take quite long.
- CLRs are read during the Analysis phase and redone during the Redo phase.
- Consider the recovery example shown in Fig. 4.18.1. There are three transactions : $Tp1$, $Tp2$, and $Tp3$. $Tp1$ updates page Z, $Tp2$ updates pages Y and Z, and $Tp3$ updates page X.

Lsn	Last_Lsn	Tran_id	Type	Page_id	Other_information
1	0	Tp1	Update	Z	...
2	0	Tp2	Update	Y	...
3	1	Tp1	Commit		...
4	Begin checkpoint				
5	End checkpoint				
6	0	Tp3	Update	X	...
7	2	Tp2	Update	Z	...
8	7	Tp2	Commit		...

Fig. 4.18.1(a)

Transaction table		
Transaction_id	Last_Lsn	Status
Tp1	2	Commit
Tp2	2	In progress

(b)

Dirty page table	
Page_id	Lsn
Z	1
Y	2

(c)

Fig. 4.18.1

The log at point of crash

Transaction table		
Transaction_id	Last_Lsn	Status
Tp1	3	Commit
Tp2	2	In progress

Dirty page table	
Page_id	Lsn
Z	1
Y	2

Fig. 4.18.2

Transaction table :

Transaction_id	Last_Lsn	Status
Tp1	3	Commit
Tp2	2	In progress
Tp3	6	In progress

Dirty page table

Page_id	Lsn
Z	1
Y	2
X	6

Fig. 4.18.3

- Fig. 4.18.1 shows the partial contents of the log, and Fig. 4.18.2 shows the contents of the Transaction Table and Dirty Page Table. Now, consider that a crash occurs at this point.
- Since there is checkpoint, the address of the related begin_checkpoint record is accessed, which is location 4. The start of analysis phase is from location 4 until it reaches the end.
- The end_checkpoint record would contain the Transaction Table and Dirty Page Table in Fig. 4.18.2, and in the analysis phase these tables are reconstructed. When the analysis phase finds log record 6, a new entry for transaction $Tp3$ is made in the Transaction Table and a new entry for page X is made in the Dirty Page Table.
- After log record 8 is analyzed, the status of transaction $Tp2$ is changed to commit in the Transaction Table.

Fig. 4.18.3 shows the two tables after the analysis phase.

For the REDO phase, the smallest LSN in the Dirty Page Table is 1. Hence the REDO will start at log record 1 and proceed with the REDO of updates. The LSNs {1, 2, 6, 7} corresponding to the updates for pages Z, Y, X, and Z, respectively, are not less than the LSNs of those.

So those data pages will be read again and the updates reapplied from the log. At this point, the REDO phase is finished and the UNDO phase starts. From the Transaction Table, UNDO is applied only to the active transaction Tp_3 .

The UNDO phase begins at log entry 6 (the final update for Tp_3) and proceeds backward in the log. For transaction Tp_3 the backward chain of updates (in this example the log records 6) is followed and undone.

Checkpoints

- To avoid rescanning the whole log file during the analysis phase, it is advisable to save the DPT and the TT regularly to the log file, forming a checkpoint.
- Instead of having to run through the whole file it is just necessary to run backwards until a checkpoint is found.
- From that point it is possible to restore the DPT and the TT as they were at the time of the crash by reading the log file forward again.
- Then it is possible to proceed as usual with Redo and Undo.
- The naive way for check pointing involves locking the whole database to avoid changes to the DPT and the TT during the creation of the checkpoint.
- Fuzzy logging avoids that by writing two log records.
- Between the two records other log records can be created. During recovery it is necessary to find both records to obtain a valid checkpoint.

Chapter Ends...

□□□